

Kurve RSC

Learning Relational Representations with the Downstream Predictor in the Loop

Kurve AI

August 27, 2026

Contents

1	Abstract	1
2	The central idea	2
3	Why frame shape is part of learning	2
4	The two-stage learning protocol	3
4.1	Stage 1: search on a controlled point-in-time frame	3
4.2	Stage 2: rebuild the winner as a production artifact	4
5	The downstream learner is a replaceable participant	4
6	Kurve RSC and GraphReduce: a deliberate boundary	4
7	Evaluation context and attribution	5
8	What Kurve RSC is optimizing—and what it is not	5
9	Conclusion	6
10	References	6

1 Abstract

Kurve RSC is a validation-guided system for learning how a relational database should be compressed into a model-ready table. It does not assume that one fixed flattening of a database is best. Instead, it proposes alternative relational programs—different graph depths, feature-family stages, and annotation policies—executes each program, fits a downstream learner on the resulting frame, and uses held-out predictive performance to decide which representation should survive.

This makes feature construction and model fitting one integrated learning process. The relational configuration changes the shape and meaning of the training frame; the learner reveals whether that representation carries useful signal for the task. In the current implementation, CatBoost is the bundled downstream learner and classification and regression candidates are selected by validation AUROC and MAE, respectively. The dataframe boundary is deliberately learner-neutral. TabPFN and other tabular learners are planned extensions, not requirements of the architecture.

Kurve RSC is built with [GraphReduce](#), the open-source engine that executes relational compute graphs. Kurve RSC is the larger learning and model-lifecycle system around that engine: it defines the candidate space, binds tasks and cutoffs, compares the resulting representations through a downstream learner, selects a configuration, freezes the relational execution plan and feature schema, and produces a fitted artifact for validation and inference.

This white paper focuses on that optimization idea. The companion [Kurve RSC technical report](#) contains the detailed treatment of cutoff semantics, relational reduction, feature families, system budgets, benchmark protocol, and reproducibility.

2 The central idea

Most relational machine-learning pipelines make two decisions in sequence: first, an engineer fixes a flattening strategy; then, a model is optimized on the resulting table. Kurve RSC moves the first decision inside the validation loop.

Let a relational configuration be

$$q = (d, \mathcal{A}, a, b),$$

where d is graph depth, $\mathcal{A}$ is the cumulative set of enabled feature families, a is the automatic-annotation policy, and b denotes feature budgets. For a database $\mathcal{D}$, task rows L , and cutoff c , GraphReduce executes the relational program selected by q and Kurve RSC materializes the entity-grain frame

$$F_q(c) = \text{Materialize}(\mathcal{D}, L, c; q).$$

Kurve RSC then fits a downstream learner M_q on the training portion of that frame and evaluates it on the validation portion:

$$M_q = \text{Fit}(F_q^{\text{train}}), \quad J(q) = \text{Metric}(M_q, F_q^{\text{validation}}).$$

The winning relational representation is therefore the one supported by the performance of a fitted model, not the one that merely creates the most features:

$$q^* = \begin{cases} \arg \max_q J(q), & \text{classification AUROC,} \\ \arg \min_q J(q), & \text{regression MAE.} \end{cases}$$

This is joint in the systems sense: representation construction and learner evaluation form one objective loop. It is not an end-to-end differentiable graph in which gradients pass through SQL. Kurve RSC performs deterministic, validation-guided configuration search over complete feature-and-model trials.

3 Why frame shape is part of learning

Every candidate is a hypothesis about which relational signal should reach the prediction grain.

A shallow, **base**-only candidate may reduce only nearby tables and emit a narrow frame containing stable schema-aware rollups. Increasing depth admits longer foreign-key paths, so additional tables can contribute summaries to the root entity. Adding **temporal** introduces windowed numeric behavior. Adding **conditional** introduces bounded, condition-specific composition. Switching automatic annotations changes the predicates and gated statistics available to those reductions.

For candidate q , write the model-facing frame shape as

$$F_q \in \mathbb{R}^{n_q \times p_q}.$$

The row count n_q is chiefly determined by the eligible labeled entities and cutoffs. The feature count p_q and the semantic lineage of those columns are functions of depth, reachable tables, family selection, annotation policy, source types, and available values. Graph execution or label attachment can also make row counts differ in practice, so Kurve RSC records both dimensions for every completed trial.

The important point is that these are not cosmetic transformations of one fixed matrix. A depth-1 **base** frame and a depth-3 **base + temporal + conditional** frame are different representations of the task. Wider is not automatically better: extra columns can be redundant, sparse, expensive, or easier for the learner to overfit. Validation performance is the feedback that resolves that ambiguity.

The current default candidate schedule is deterministic and cumulative:

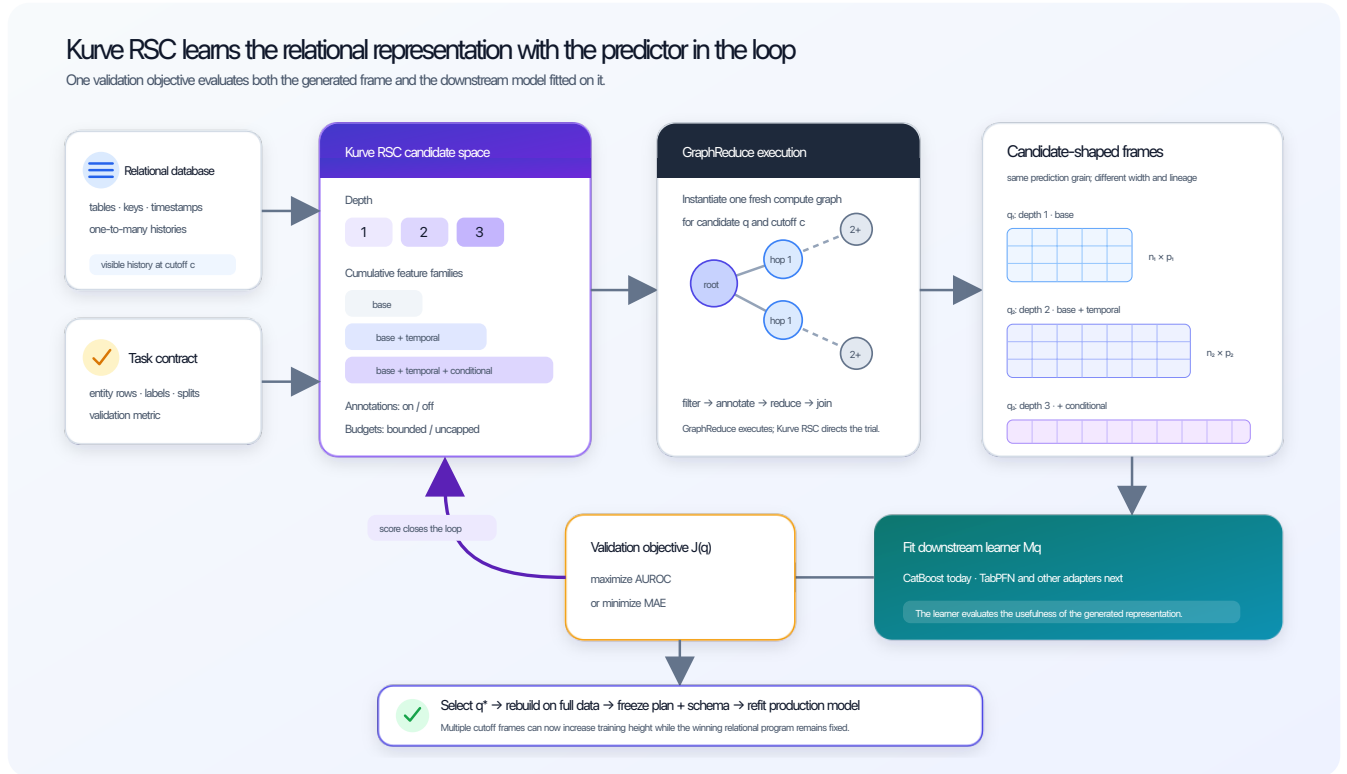


Figure 1: Kurve RSC jointly evaluates relational configuration and downstream model performance. Each candidate changes graph reach and feature policy, producing a differently shaped frame before learner fitting and validation scoring.

Search axis	Current default
Relational depth	1, 2, and 3 hops back toward related tables
Feature-family stage	base; base + temporal; base + temporal + conditional
Automatic annotations	enabled and disabled
Downstream learner	CatBoost
Classification objective	maximize validation AUROC
Regression objective	minimize validation MAE

The cross-product produces 18 default representation candidates. The stages are cumulative so each expansion has an interpretable relationship to the smaller program that preceded it. Users can supply an explicit set of `GraphConfig` candidates when the default space is not appropriate.

4 The two-stage learning protocol

Kurve RSC separates economical configuration discovery from the production fit. This distinction matters because relational materialization may be more expensive than fitting the downstream model itself.

4.1 Stage 1: search on a controlled point-in-time frame

Each candidate receives a fresh GraphReduce graph because graph execution is stateful. The current implementation caps each search source at a configurable sample size (100,000 rows by default) and searches on the latest eligible training cutoff. For every candidate, Kurve RSC:

1. builds the depth-bounded relational graph and applies the candidate’s feature policy;
2. learns a feature-operation plan from training data;

3. materializes aligned training and validation frames under that plan;
4. normalizes model inputs while freezing the training-derived column order;
5. fits CatBoost and records validation score, feature count, row counts, feature time, and model time; and
6. preserves failures and resource-related skips in the trial audit trail.

The configuration with the best objective score becomes `best_config`. Kurve RSC also exposes `recommended_config`: the simplest candidate within a small performance tolerance of the best. By default, a classification candidate is eligible when it is within 0.002 AUROC of the winner; a regression candidate is eligible when its MAE is within 0.5% of the winner. This makes the cost of representational complexity visible without replacing the actual metric winner.

4.2 Stage 2: rebuild the winner as a production artifact

Search chooses a configuration; it does not make the sampled search graph the production model. Kurve RSC rebuilds only the winner from uncapped sources. It learns one exact GraphReduce execution plan on full training data, freezes that plan and its ordered feature schema, replays both on validation data, and records the full-data validation score.

For timestamped problems, the production refit can use several point-in-time training frames. If the selected cutoffs are $c_1, \dots, c_K$, their compatible frames are row-unioned:

$$F_{q^*}^{\text{train}} = F_{q^*}(c_1) \oplus \dots \oplus F_{q^*}(c_K).$$

Here K increases temporal coverage and usually frame height, while d and $\mathcal{A}$ control relational reach and feature width. The operation plan is learned once on the latest full-training frame and replayed at the other cutoffs, preventing each historical view from silently inventing a different feature program.

After validation, the production CatBoost model is refit on full training plus validation. Test or future frames replay the frozen graph plan and training-only schema with `GraphReduce(train=False)`. Feature discovery does not run again at inference time.

5 The downstream learner is a replaceable participant

Kurve RSC’s stable interface to learning is an aligned dataframe, target, split contract, and metric. This is intentionally narrower than a model framework and broader than a CatBoost-specific feature generator.

CatBoost is the primary backend today because it is a strong conventional learner for mixed numerical and categorical data and can expose the value of a relational representation without requiring a bespoke neural architecture. The implementation currently holds the supplied CatBoost parameter policy fixed while it searches the relational configuration; CatBoost early stopping may adapt the fitted iteration count on sufficiently large validation sets.

TabPFN is a natural next backend. From Kurve RSC’s perspective, the relational system compiles database history into the tabular context on which a foundation model can condition or be fine-tuned. Context length, row budget, and feature budget will affect the feasible candidate space, but they do not change the contract: a candidate is judged after the chosen learner consumes its frame. The same separation can support other gradient-boosted trees, linear models, neural tabular models, or AutoML systems as adapters are added.

Learner neutrality should not be confused with implemented parity. The current packaged path is CatBoost; TabPFN and additional learners are forward work. Kurve RSC’s architectural contribution is that adding them does not require redefining point-in-time relational construction.

6 Kurve RSC and GraphReduce: a deliberate boundary

Kurve RSC and GraphReduce are related projects with distinct responsibilities. GraphReduce deserves direct credit as the relational execution engine on which the current Kurve RSC implementation is built. Kurve RSC should nevertheless be understood as its own learning system, not as a new name for GraphReduce.

Kurve RSC owns the learning system	GraphReduce supplies the execution substrate
Task and schema adaptation	Table-node and relationship-edge abstractions
Candidate-space construction	Ordered relational compute operations
Depth, family, annotation, and budget search	SQL generation and execution
Cutoff scheduling and entity-frame attachment	Temporal filtering primitives
Sampled search and full-data winner rebuild	Many-to-one reduction and feature propagation
Validation objective and complexity recommendation	Reusable feature-family implementation
Frozen-plan/schema lifecycle and prediction API	Compute-graph runtime machinery
Downstream learner integration and fitted artifact	Execution of the relational program requested by Kurve RSC

A useful analogy is that GraphReduce is the compiler/runtime for a relational program, while Kurve RSC decides which programs to propose, runs the empirical selection process, and binds the winning program to a predictive model and its lifecycle. The value of Kurve RSC lies in this orchestration and learning loop: it converts graph configuration from an engineering constant into a validation-tested hypothesis.

The boundary also makes both projects more useful. GraphReduce remains a general-purpose relational feature-engineering project that can be used without Kurve RSC. Kurve RSC can evolve its search policy, task adapters, model backends, and experiment semantics while continuing to rely on a specialized relational engine.

7 Evaluation context and attribution

This white paper intentionally contains no performance table. The detailed benchmark report presents Kurve RSC classification and regression results, run disclosures, ablations, and caveats. Readers should consult that report before drawing empirical conclusions.

For classification context, the external comparison columns reproduced in the technical report originate from **Appendix D, Table 4 of the August 2026 Prior Labs RelArena- α report** [6]. Credit for that table, its RelArena evaluation work, and the reported TabPFN-Rel, RDBLearn, RelGNN, and RelGT results belongs to Hayler, Flöge, Arazi, Ranjan, Leskovec, Purucker, Hutter, Hollmann, and the Prior Labs Team. Those external values were not produced by the Kurve RSC repository.

RelArena comparisons are valuable context, but they are not automatically a controlled within-harness experiment. Package revisions, database state, tuning budget, cutoff treatment, and execution protocol can differ. Kurve RSC therefore treats validation discipline and full run disclosure as part of the result, not as footnotes to it.

RelBench supplies the official relational datasets, task definitions, temporal splits, and evaluators used by these experiments [3]. Kurve RSC does not redefine those benchmark labels or metrics; it learns a representation and predictor for the task contract RelBench provides.

8 What Kurve RSC is optimizing—and what it is not

Kurve RSC optimizes the predictive utility of a relational representation under a declared search space and validation split. In the current system, that includes depth, cumulative feature-family stage, automatic annotations, and optional family column budgets. The objective reflects the combination of the generated frame and fitted learner.

It does not yet claim to solve every related optimization problem:

- Search is finite and staged, not continuous or gradient-based.
- The default search does not jointly sweep arbitrary CatBoost hyperparameters or learner families; caller-supplied model parameters are held fixed across relational candidates.
- Search currently uses one latest sampled training frame, while the winning configuration may be rebuilt over multiple full-data cutoff frames.
- The primary selection objective is predictive performance. Runtime and feature count inform the simpler recommendation but are not yet a general multi-objective cost optimizer.
- Validation can select among proposed configurations only; it cannot recover useful signal excluded from the candidate space or unavailable under the declared point-in-time history.

These constraints are purposeful. They make each trial auditable and keep test labels outside configuration choice. They also identify a clear roadmap: adaptive candidate generation, cost-aware multi-objective search, learner-aware feature budgets, multiple downstream backends, and policies that decide how many full-training frames are worth materializing.

9 Conclusion

Kurve RSC learns more than model parameters. It learns which relational representation should be presented to the model.

Each candidate configuration reaches a different distance into the database, preserves a different family of signals, and produces a frame with its own shape and lineage. A downstream learner is fitted on that frame, and held-out performance closes the loop. The selected representation is then rebuilt on full data, frozen as an execution plan and schema, and carried into a production fitted artifact.

GraphReduce is the enabling relational engine and an important open-source foundation for this work. Kurve RSC is the independent system that turns that engine into a validation-guided representation-learning process coupled to an interchangeable downstream learner. CatBoost is the practical starting point; TabPFN and other learners extend the same idea. The enduring abstraction is the loop itself: propose a relational view, learn on it, validate it, and keep the view whose usefulness survives contact with the prediction task.

10 References

1. W. Madrigal. **GraphReduce: relational feature engineering through graphs of tables, keys, and compute operations**. Open-source software project. [GitHub](#), [documentation](#), and [PyPI](#).
2. Kurve AI. **Kurve RSC: Multi-Cutoff Relational Signal Compression, Downstream Learning, and Kurve RSC Feature Families**. Technical report, August 2026. [Companion report](#).
3. J. Robinson, R. Ranjan, W. Hu, K. Huang, J. Han, A. Dobles, M. Fey, J. E. Lenssen, Y. Yuan, Z. Zhang, X. He, and J. Leskovec. **RelBench: A Benchmark for Deep Learning on Relational Databases**. NeurIPS Datasets and Benchmarks, 2024. [arXiv:2407.20060](#).
4. L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin. **CatBoost: Unbiased Boosting with Categorical Features**. NeurIPS, 2018. [CatBoost paper references](#).
5. N. Hollmann, S. Müller, L. Purucker, A. Krishnakumar, M. Körfer, S. B. Hoo, R. T. Schirrmeister, and F. Hutter. **Accurate Predictions on Small Data with a Tabular Foundation Model**. *Nature* 637, 319–326, 2025. [doi:10.1038/s41586-024-08328-6](#).
6. A. Hayler, K. Flöge, A. Arazi, R. Ranjan, J. Leskovec, L. Purucker, F. Hutter, N. Hollmann, and the Prior Labs Team. **Advancing Open and Reproducible Relational Learning: RelArena- α , TabPFN-Rel and RPI**. 2026. [arXiv:2608.16319](#) and [official RelArena results snapshot](#).
7. Prior Labs Team. **TabPFN-3: Technical Report**. May 12, 2026. [Prior Labs technical report](#).